EECS ELECTRICAL ENGINEERING AND COMPUTER SCIENCE
UNIVERSITY OF MICHIGAN

PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU

FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU

Oct 28, 2025

Tongyuan Miao, Gary Huang, Kai Jun Han, Annie Jiang

PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU

Institute of Parallel and Distributed Systems (IPADS), Shanghai Jiao Tong University
SOSP 2024

Introduction

There’s growing demand for LLM inference on consumer-grade GPUs

- No reliance on big cloud / proprietary companies
- Better data privacy
- Better accessibility
- Better customization / freedom

Main issue: consumer-grade GPUs are built for video games

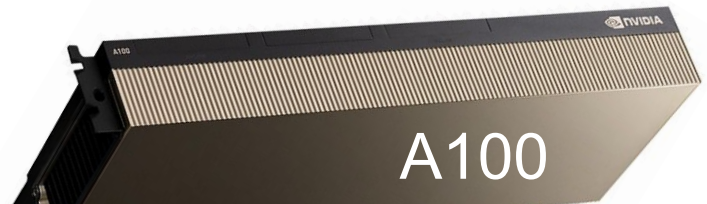
Introduction

Main issue: consumer-grade GPUs are built for video games

- They have too much (FP32) compute for their own good
- Not enough memory (and bandwidth) for *traditional* LLM inference

~$2000

Texture Rate:	1,290.2 GTexel/s
FP16 (half):	82.58 TFLOPS (1:1)
FP32 (float):	82.58 TFLOPS
FP64 (double):	1,290.2 GFLOPS (1:64)
Memory Type:	GDDR6X
Memory Bus:	384 bit
Bandwidth:	1.01 TB/s



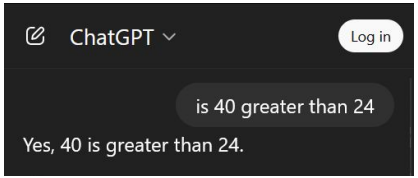
~$20000

TF32:	155.92 TFLOPs (8:1)
FP64 Tensor:	19.49 TFLOPS (1:1)
FP16 (half):	77.97 TFLOPS (4:1)
FP32 (float):	19.49 TFLOPS
FP64 (double):	9.746 TFLOPS (1:2)
Memory Size:	80 GB
Memory Type:	HBM2e
Memory Bus:	5120 bit
Bandwidth:	1.94 TB/s

https://www.techpowerup.com/gpu-specs/

Background (Motivation for Offloading)

Main issue: consumer-grade GPUs are built for video games

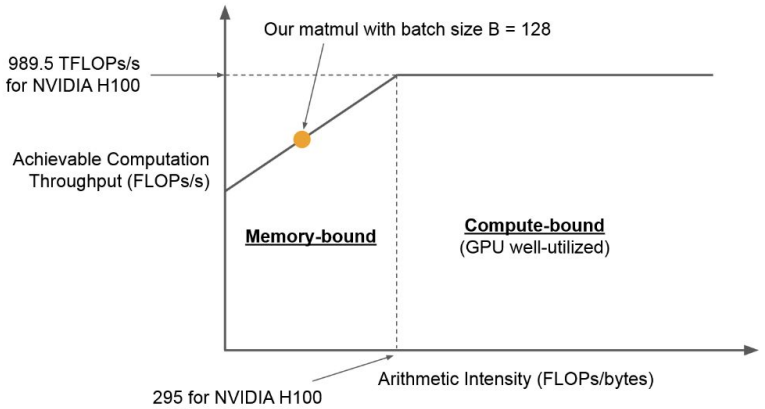


LLM inference requires lots of memory capacity and bandwidth

- OPT-66B with 4-bit quantization requires 40GB just for parameters
- Model must be distributed between GPU and CPU memory (*offloading*), causing significant overhead via PCIe
- Selecting what to put in GPU memory is crucial to alleviating that overhead

This is a *memory* problem

This is a ***locality*** problem



From Prior Lecture (Jae-Won Chung, 2025)

Background (Current Offloading Solutions)

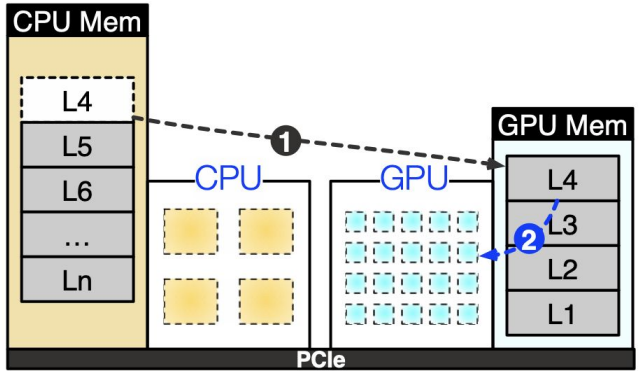
Existing offloading solutions divide work by layer

GPU-centric offloading: parameters are transferred from CPU to GPU as they are processed

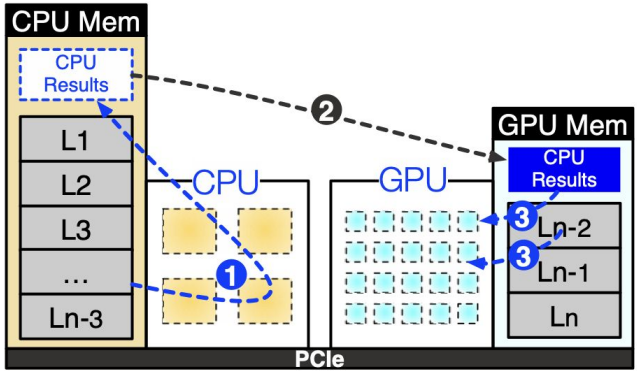
- FlexGen uses zig-zag scheduling, prioritize throughput over latency
- DejaVu suffers from frequent transfer from CPU to GPU during runtime

GPU-CPU hybrid offload: CPU processes each layer before sending to GPU for token generation

- llama.cpp has reduced latency but still suffers from poor locality and constrained memory capacity, performing most computation on CPU

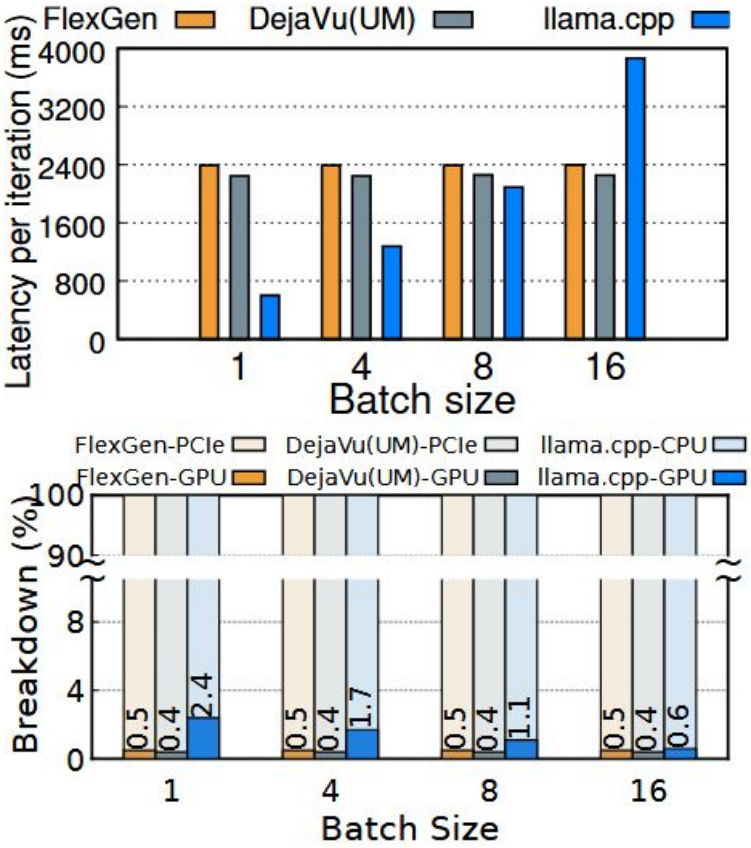


(a) GPU-Centric Offloading



(b) GPU-CPU Hybrid Offloading

Background (Current Offloading Solutions)



GPU-centric offloading: FlexGen, DejaVu
GPU-CPU hybrid offload: llama.cpp

FlexGen spends 99.5% of the time transferring parameters from CPU to GPU

llama.cpp has best latency (~600ms), still very slow compared to A100 (~45ms)

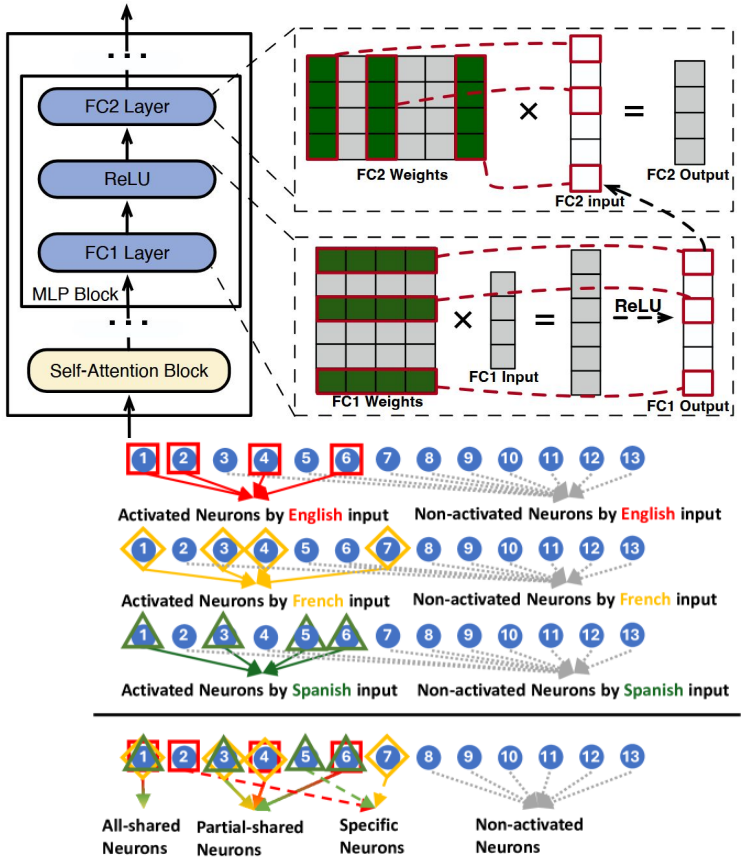
llama.cpp handles 98% of computational time with CPU, as GPU only has capacity for 37% of the OPT-30B model

Background (Activation Sparsity)

Not all neurons are created equal

- After the series of matrix multiplications, the result is passed through an non-linear *activation function*
- A neuron is **activated** if the output is *non-zero*
- Depending on the input, only a subset of all neurons is activated

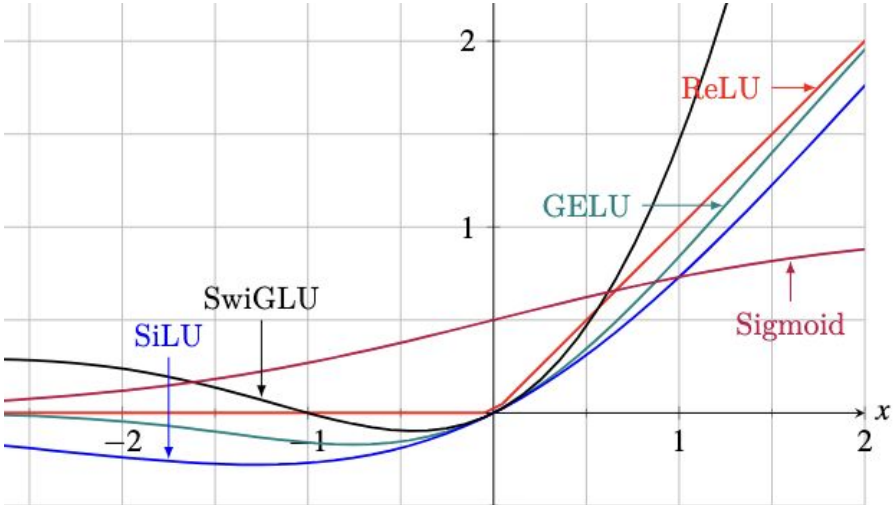
Dilemma: We want only the neurons that will activate on the GPU, but we only know if a neuron activated after computing its result



Weixuan Wang, et al, 2024

Background (Activation Sparsity)

LLM	Activation Function	Sparsity
OPT-30B	ReLU	97%
LLaMA2-13B	SwiGLU	43%
Yi-34B	SwiGLU	53%



LLM inference exhibits notable sparsity

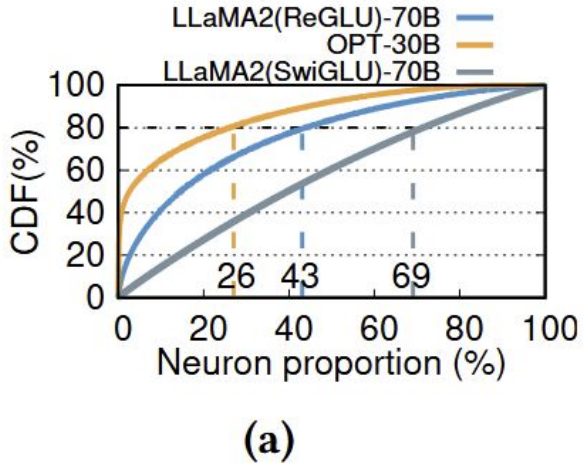
- Half of the heads in self-attention contribute minimally, while MLP sparsity depends on activation function
- DejaVu uses online predictor to only process activated neurons with 93% prediction accuracy and 6x speedup
- Findings further align with results from CATS and ReLU2

https://machinelearningmastery.com/linear-layers-and-activation-functions-in-transformer-models/

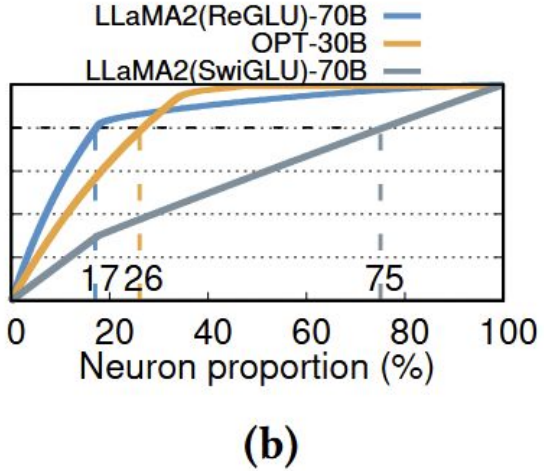
Observations : Power Law

Power-Law Activation Distribution

- Within an MLP layer, only a fraction of the neurons are responsible for 80% of the activation
- Across the Entire Model, this high locality is being reflected as well



(a)



(b)

Observations : Power Law

Extreme Cases : OPT - 30B

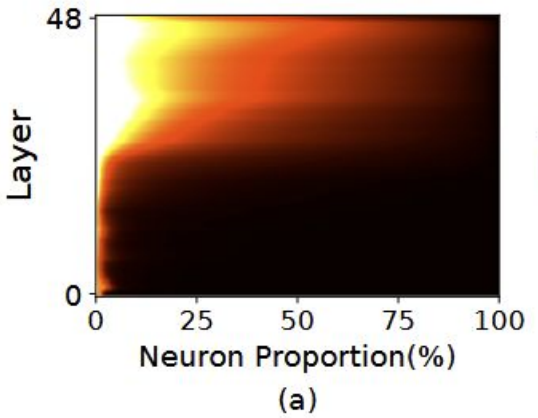
- In the initial 24 layers, OPT-30B has <1% neurons becoming active

Realistic Scenario:

- In each individual iteration, ~10% of neurons are activated (Note: This is true for relu-family llms)
- Other functions only ~50% sparsity (i.e. SwiGLU)

Realistic Scenario:

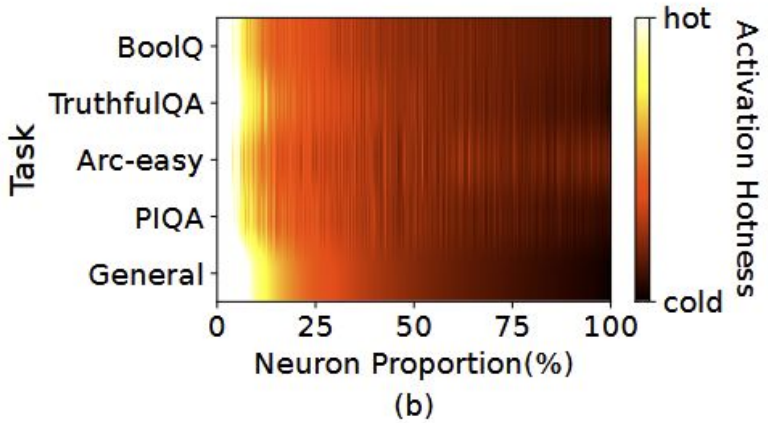
- More people are directly training LLMs with RELU
- This is False!



Observations : Power Law

Cross Domain Compatibility

- hot neurons are consistently activated consistently across different tasks
- The power-law pattern in neuron activations stems from the model's architecture itself, not from any particular dataset

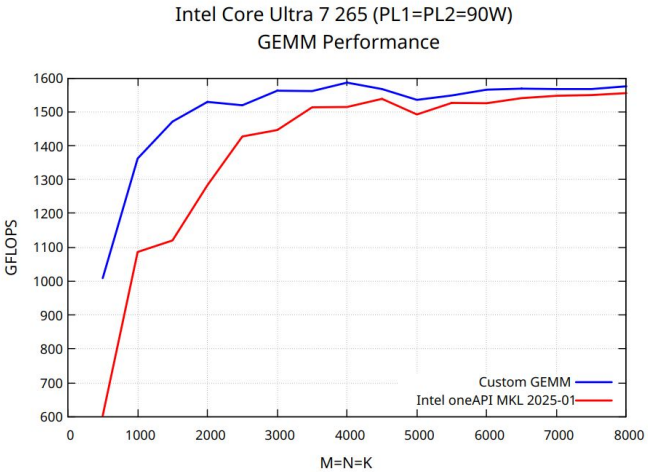


(b)

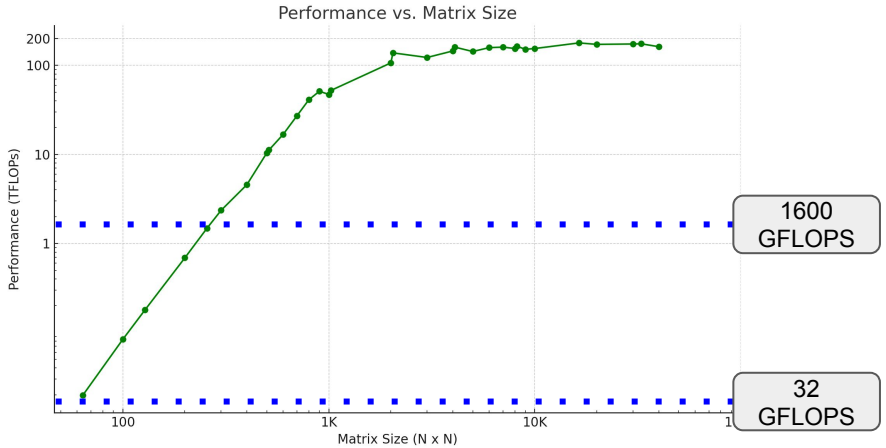
Observations : CPUs are not that slow

AVX2

- a 256-bit x86 SIMD instruction set extension that adds integer vector operations to accelerate parallel data processing.
- Available in all Intel CPUs 2013 and newer and AMD CPUs 2015 and newer
- RTX 4090 : PCIe 4.0 x16 : 32 GB/s (paper said 64?)-> on the order of 16-32 GFLOPS



Source: A. Salykov, 2025



RunPod 1x RTX4090, FP16, PyTorch 2.0.1
Source: Reddit

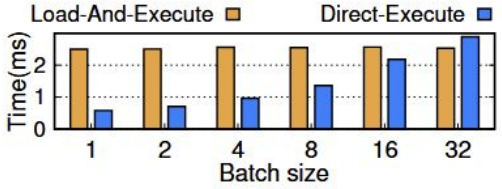
Observations : Memory loading can be slower

Memory Bandwidth

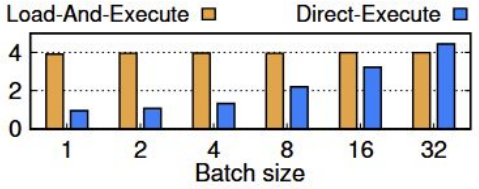
- RTX 4090 : PCIe 4.0 x16 : 32 GB/s (paper said 64?)-> on the order of 16-32 GFLOPS
- CPU (i9-13900K) : 67.2 GB/s (Maximum 89.6 GB/s (Source: Intel)) (compute bound)
- Footnote: CPUs can go even higher, AMD EPYC can go up to 614 GB/s (AMD EPYC 9965)

Testing:

- 10% sparsity of MLP block and 60% sparsity of Attention block (Load then compute vs CPU compute)
- For Batch Size (#prompts/requests) < 32, CPU wins!



(a) MLP block

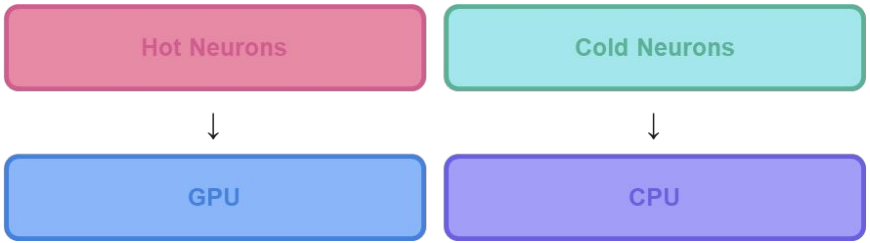


(b) Attention block

PowerInfer Overview

Main Strategy

- Put HOT neurons on fast GPU (preloaded prior inference)
- Keep COLD neurons on CPU
- Compute only ACTIVATED neurons

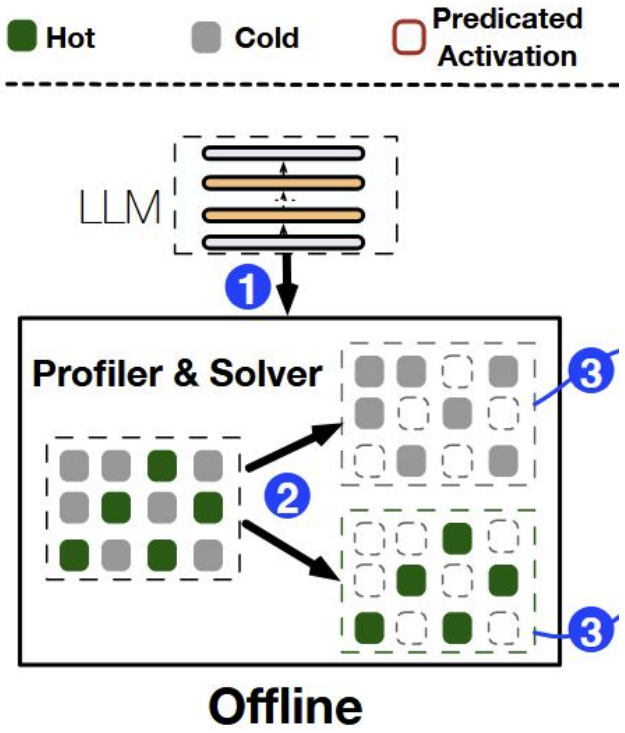


Not from paper, self-drawn

PowerInfer Internals : overview

Offline profiler -> Observation 1

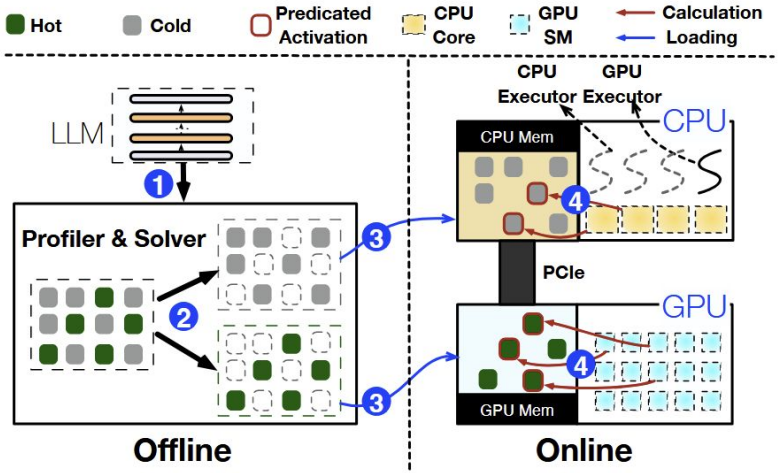
- Profile neuron activation with general datasets (C4)
- Solve placement policy
- Generate neuron maps



PowerInfer Internals : overview

Example

- assigns neurons to their respective processing units (3)
- Engine predicts runtime hot neurons (4)
- Create executor to manage concurrent CPU-GPU computations (4)
- CPU transfer results back to GPU for integration



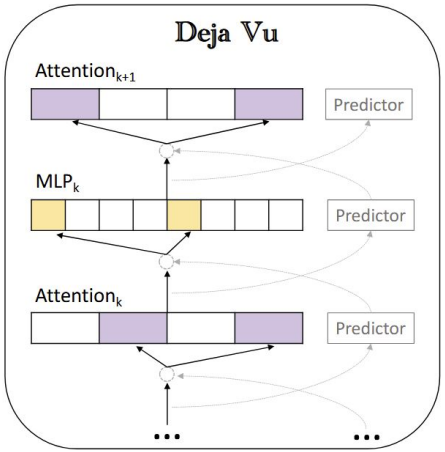
PowerInfer Internals : Inference Engine Dive In

Déjà vu? DejaVu!

- Key IDEA: Only compute neurons that will be activated
- 2 predictors: attention and MLP each

Limitations

- Runtime predictions are challenging (you have a whole model still running!)
- These predictors are not small
  - 27GB of GPU memory for OPT-175B
  - Predictor larger than the 4090
  - Naively reducing predictor size leads to large performance drop



Source: Z. Liu et al. 2023

PowerInfer Internals : Inference Engine Dive In

Do you need that big of a model?

- The sparser it is, the easier it is
- The skewer it is, the easier it is

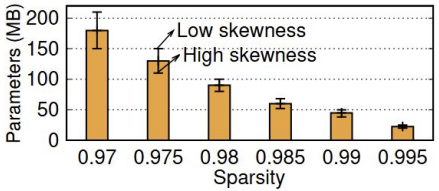


Figure 9. Correlation between predictor parameter size and layer sparsity at a guaranteed 95% accuracy level for OPT-175B. The

Solution

- A “non-fixed-size” MLP predictor for each layer (benchmarked on WikiText-2)
  - Fixed input, fixed output, flexible hidden
- Starts with a baseline size
  - Iteratively adjusting the size, based on the skewness and sparsity
  - achieved a discrepancy of less than 0.1% (w.r.t baseline)
- Result: predictor is only 6% of total model size

PowerInfer Internals : Neuron Aware Operators

Compiler Challenges

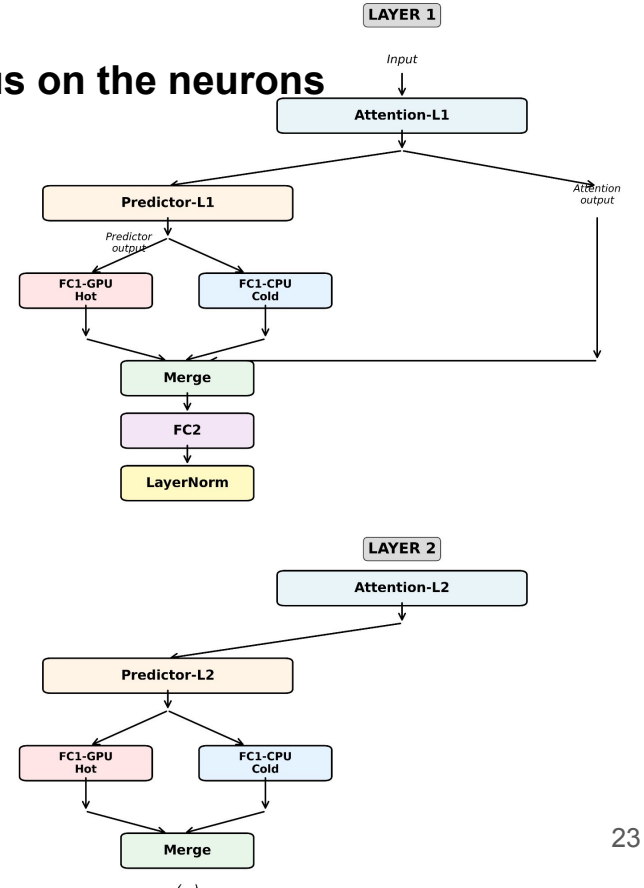
- SOTA sparse-aware compilers do not work well
  - Static compilers do not work (SparTA & FlashLLM)
  - Others are dynamic, but converts back to dense (cuSPARSE)
    - Large overhead
- JIT compiler?
  - PIT works well on GPU but does not work on CPU well

PowerInfer Internals : Neuron Aware Operators

Solution: Do not treat it as a sparse matrix problem! Focus on the neurons

Custom Designed Operators

- Traditional Operators : focus on row, col of matrix
- Neuron Aware Operators:
  - determine activation status
  - If active, execute individual with row, col data

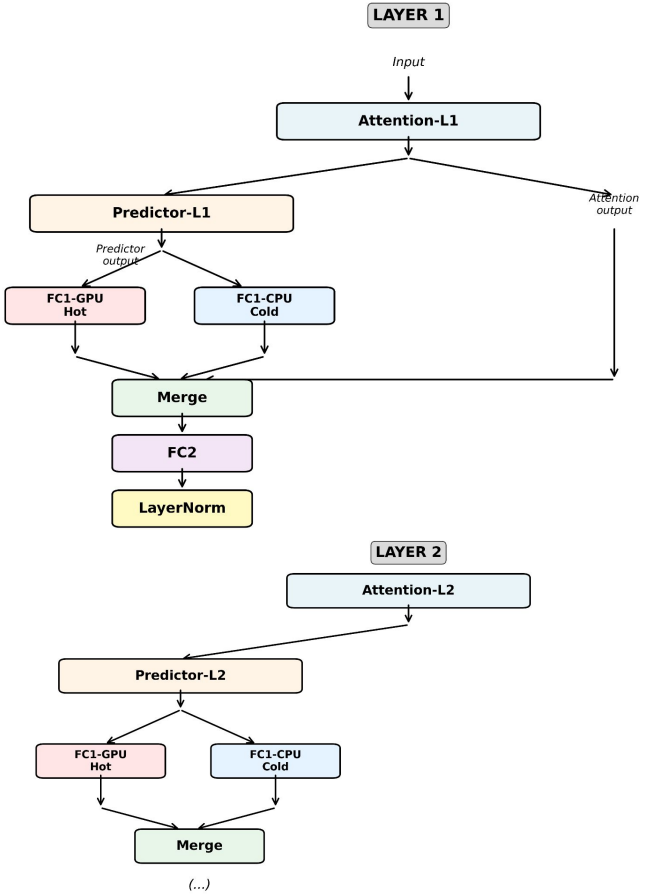


Not from paper, self-drawn

PowerInfer Internals : Neuron Execution Control

Operator Scheduling

- Build a direct-acrylic-graph
  - Each node is an inference operator
  - Store inference operator in global queue
  - Executors pull out operations, add them to appropriate processing units
  - Scheduler handle dependencies and GPU kernel launch



PowerInfer Internals : NAO Evaluation

- Neuron-aware Operators for GPU
  - No more mat-vec, just vec-vec (less efficiency?)
  - In fact, in small batches, vec-vec computation avoid sparsity overhead
  - Divergence Handling: different tasks go to different thread blocks
    - Side Note: GPU Partitioning is not as good as it seems
- Neuron-aware Operators for CPU
  - NAO is less parallel than matrix (Good for CPU!)
  - Multi Core-> Each Core process a task
  - AVX2 can handle vec-vec operations well

Neuron Placement Policy

Symbol	Type	Description
$\mathbb{N}$	Par	All neurons
$\mathbb{U}$	Par	CPU and GPU
f_i	Par	Activation frequency of neuron i
N_i	Par	Neuron in layer i
v_i	Par	Neuron impact for neuron i

$$v_i = f_i \qquad \sum_{i \in \mathbb{U}} a_{in} = 1$$

$$Maximize \quad t_i = \sum_{e \in \mathbb{N}} a_{ie} * v_e \forall i \in \{GPU\}$$

Policy maximizes GPU impact while satisfying memory constraints

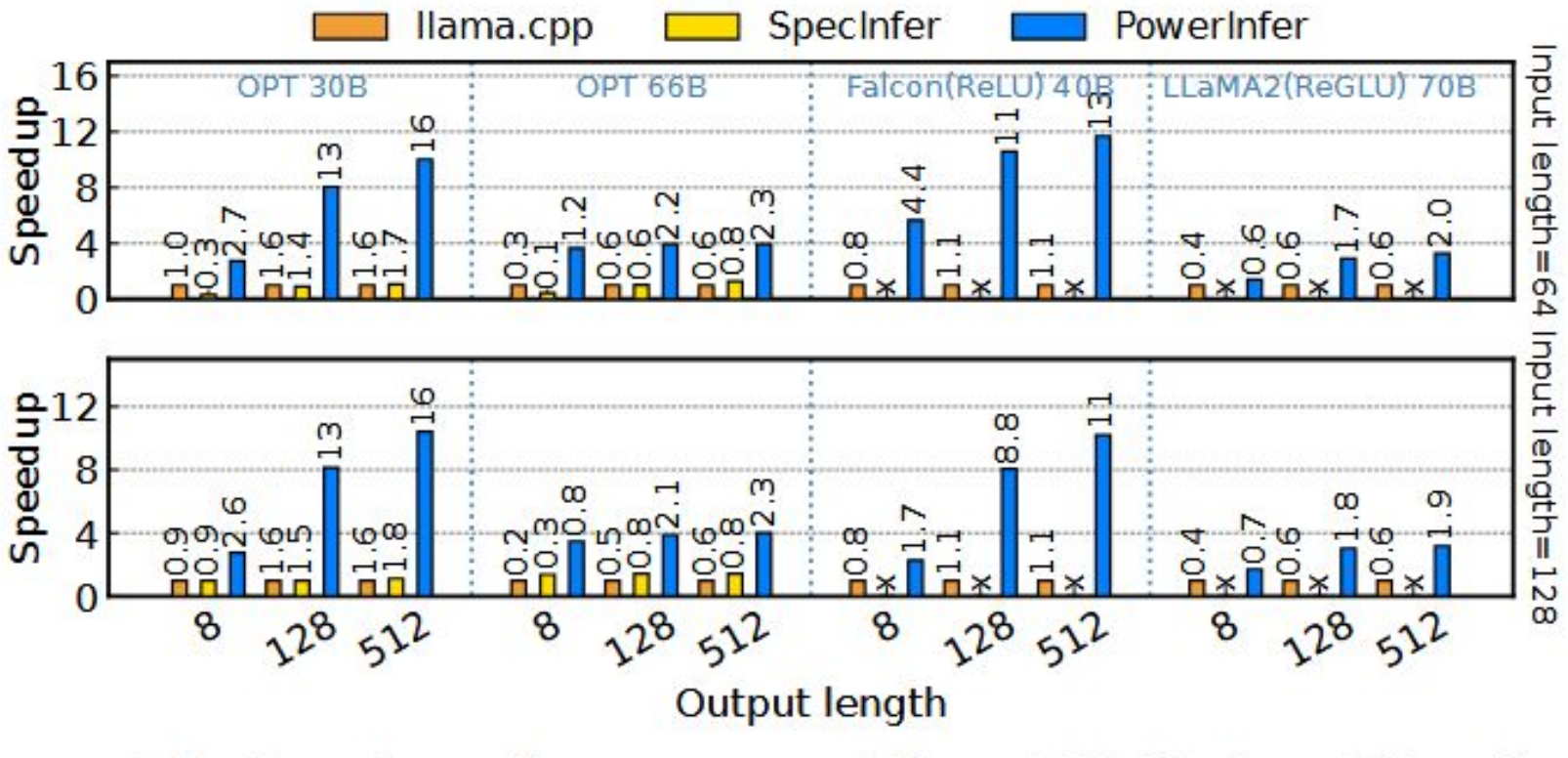
- C4 and Wikipedia were used to measure activation count of each neuron on a general corpus
- A minimum number of neurons must be place on GPU for each layer for synchronization to be worth it
- ILP solver groups 64 neurons with similar impacts, drastically reducing N and solving in 10s

Evaluation

PowerInfra architecture incorporated into llama.cpp

- **Hardware:** PC-High (Intel i9 + RTX 4090), PC-Low (Intel i7 + RTX 2080 Ti)
- **Models:** OPT (ReLU), Bamboo (dReLU), Falcon (ReLU), LLaMA2 (ReGLU), Qwen1.5 (SwiGLU)
- **Workload**: chatbot arena ChatGPT prompts and Alpaca datasets
- **Baselines:** original llama.cpp and SpecInfer (FlexGen and DejaVu not used as baseline as they have much higher latency)

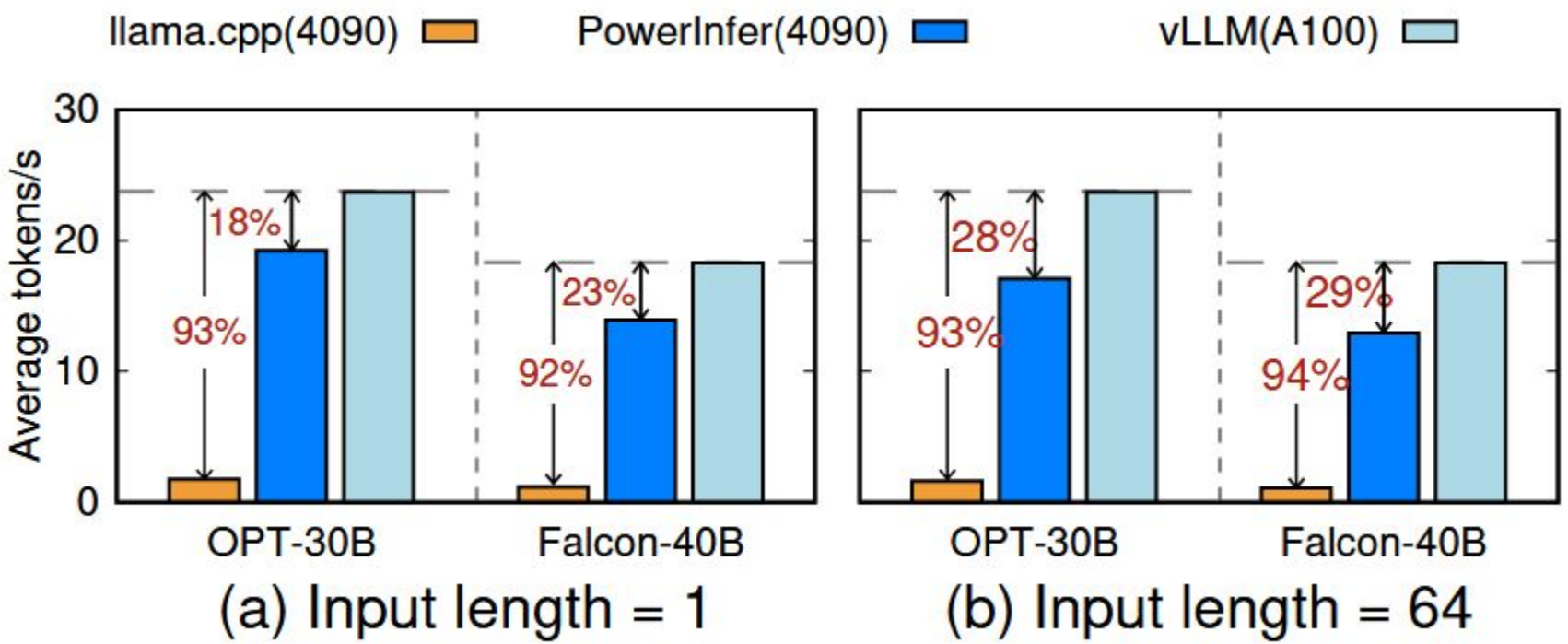
Evaluation (End-to-End)



Evaluation (LLM Accuracy)

	PIQA	Winogrande	Arc-Challenge	MMLU	GSM8K	Average
OPT-7B	75.78%	65.19%	30.63%	24.95%	1.90%	39.69%
OPT-7B-PowerInfer	75.67%	65.51%	30.63%	24.73%	1.82%	39.67%
OPT-13B	76.01%	64.96%	32.94%	25.02%	2.12%	40.21%
OPT-13B-PowerInfer	76.28%	65.98%	33.19%	24.76%	2.20%	40.48%
LLaMA(ReGLU)-13B	76.44%	70.09%	36.52%	50.21%	25.40%	51.73%
LLaMA(ReGLU)-13B-PowerInfer	74.06%	69.93%	36.60%	49.47%	23.90%	50.79%
Falcon-40B	81.23%	75.45%	50.68%	51.78%	21.99%	56.23%
Falcon-40B-PowerInfer	81.01%	75.92%	50.68%	51.68%	20.45%	55.95%
LLaMA(ReGLU))-70B	82.01%	75.93%	52.39%	62.30%	62.30%	66.99%
LLAMA(ReGLU)-70B-PowerInfer	82.05%	75.53%	51.45%	61.90%	61.90%	66.57%

Evaluation (vs. the Big Boss)



Limitations

- Lots of extra work needed for every model
  - Manual profiling to determine neuron activation frequency
  - ILP solving to generate placement policy
  - Training activation predictors is often time consuming
- ILP parameters change based on hardware configuration and specifications
- Require specially optimized AVX2 intrinsics for kernels
- How well does this architecture work with Mixture-Of-Experts models?
- Most mainstream architectures opted to use SwiGLU and not RELU

FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU

Stanford University, UC Berkeley, ETH Zurich, Yandex, HSE University, Meta, Carnegie Mellon University, International Conference on Machine Learning, 2023

Problem Statement

- LLMs have billions, trillions of parameters, requiring high computation and memory requirements
  - GPT-175B needs 325GB of GPU memory simply to load its model weights, fitting this model onto GPUs would require at least five A100 (80GB) GPUs
  - Memory footprint of LLM inference mainly comes from model weights and KV cache

Motivation

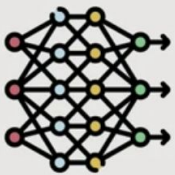
1. Making large models accessible for single-GPU systems
2. Optimizing for throughput

From  Latency-sensitive Tasks
(e.g., Chatbot)

To  Throughput-oriented Tasks
(e.g., benchmarking, information extraction, data wrangling, Text/forms processing)

From 

To 



GPT-175B

Parameter size: 325 GB

≫

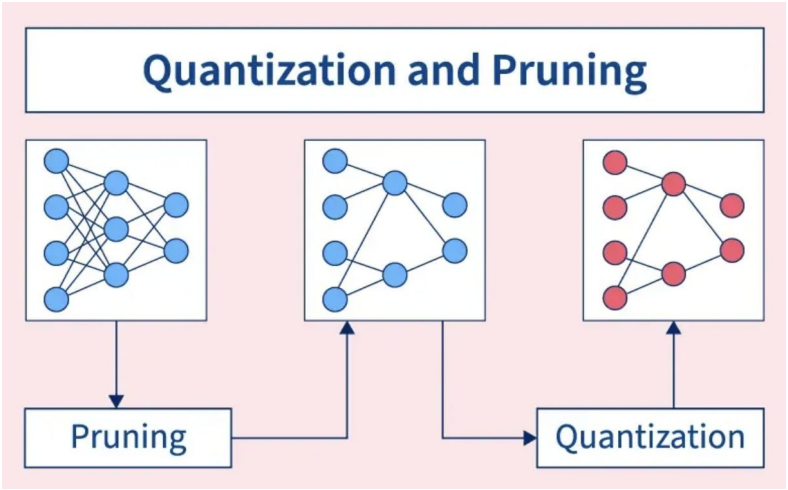
NVIDIA T4

Memory capacity: 16GB

Existing Approaches: Model Compression

Model Compression

- Goal: Reduce model size to lower memory use
- Uses quantization (fewer bits) and pruning (remove less important parameters)
- Issue
  - Works only when the compressed model fits on GPU
  - Ineffective for massive 175B models on a single GPU



Existing Approaches: Collaborative Inference

Collaborative Inference

- Decentralizes inference across multiple participants' GPUs to amortize inference cost
- Each machine runs a portion of the model to share the workload
- Amortizes computation cost across the network
- Issue
  - Assumes each GPU can still fit its assigned model part, challenging for very large models

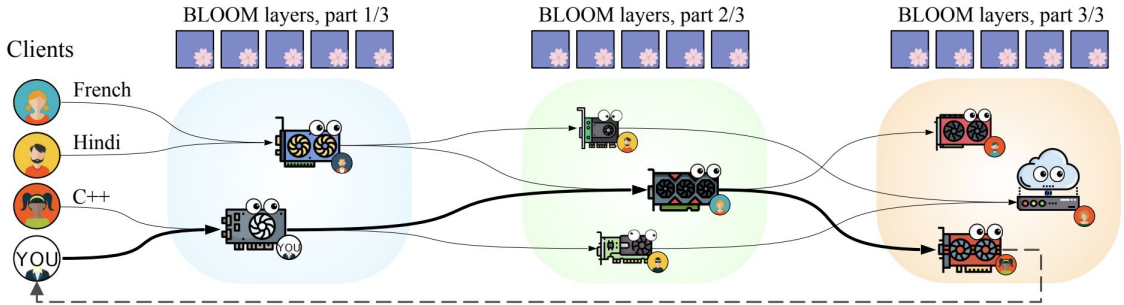
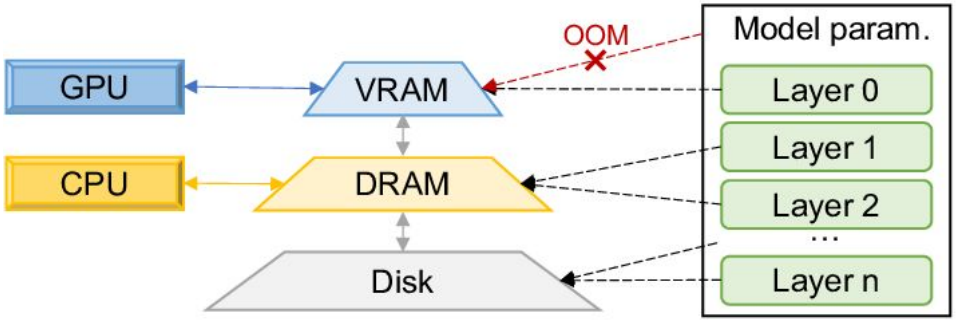


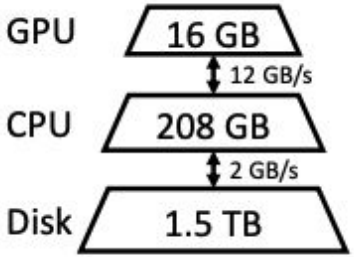
Figure 1: An overview of PETALS. Some participants (*clients*) want to use a pretrained language model to solve various tasks involving processing texts in natural (e.g., French, Hindi) or programming (e.g., C++) languages. They do it with help of other participants (*servers*), who hold various subsets of model layers on their GPUs. Each client chooses a sequence of servers so that it performs an inference or fine-tuning step in the least amount of time.

Existing Approaches: Offloading

Offloading

- Offloads model parts to CPU/disk, loading them to GPU as needed
- **DeepSpeed ZeRO-Inference**: Supports offloading the entire model weights to the CPU or disk
- **Hugging Face Accelerate**: Allows for offloading a portion of the model's weights to run on limited hardware
- Issues:
  - Inherit offloading strategies from training and have poor I/O scheduling and tensor placement
  - Both DeepSpeed and Accelerate can only put KV cache and activations on GPU, limiting batch
    - Batch size is 1-2 when running OPT-175B





Key Idea

Designing efficient offloading strategies for high throughput generative inference, on a single commodity GPU

FlexGen Overview

Offloading (moving tensors in and out of memory hierarchy)

- **Construct a search space**
  - Compute Schedule
  - Tensor Placement
- Cost model + Policy Search

Search Space: Computational Graph

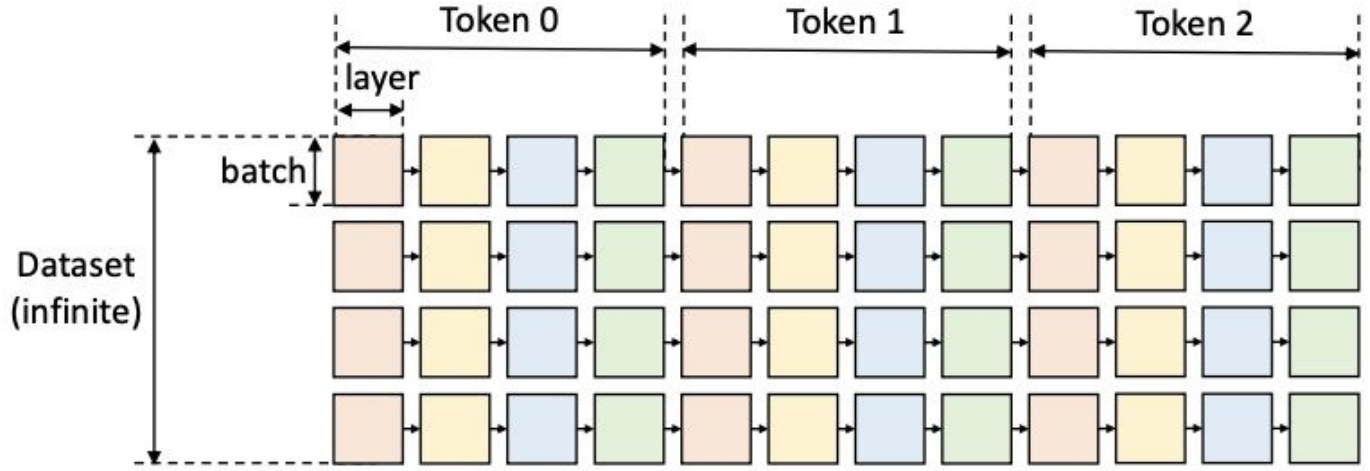
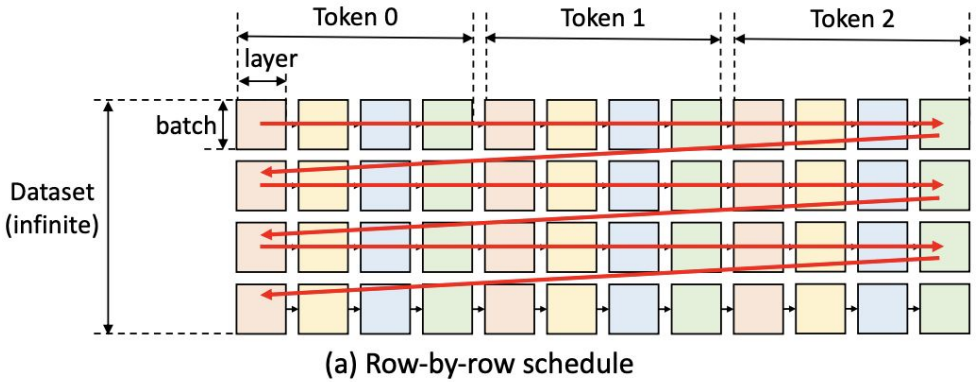


Figure 2. Computational graph of LLM inference.

- Square: the computation of a GPU batch for 1 layer
  - Squares with the same color share the same layer weights
  - To compute a square, the GPU loads the tensors it needs and offloads the cache and activations when finished
- A square can be computed only after all squares to its left in the same row are done
- Computation of square requires all inputs (weights, activations, cache) on the same device
- Each square outputs activations (kept until the next square) and KV cache (kept until the row's end)

Search Space: Row by Row Traversal



(a) Row-by-row schedule

Row-by-row traversal (used by existing systems):

- Advantages:
  - Fastest way to complete generation for one batch
  - KV cache can be freed immediately after processing each row
- Disadvantages:
  - No weight sharing between contiguous squares (rows)
  - Requires repeated loading of weights, leading to high I/O costs

Search Space: Column by Column Traversal

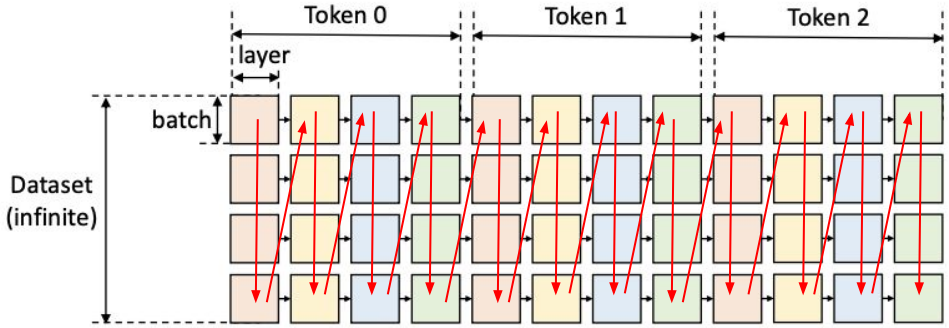
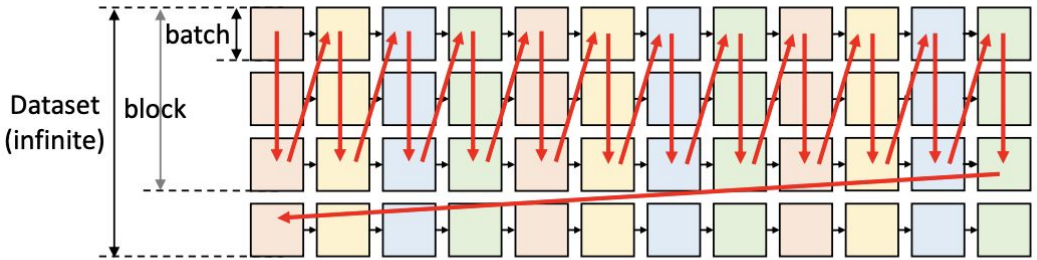


Figure 2. Computational graph of LLM inference.

Column-by-column traversal:

- All squares in a column share weights, allowing weights to stay on the GPU for reuse
- Only activations and KV cache need to be loaded or unloaded
- Limitation:
  - Cannot process an entire column continuously because activations and KV cache consume CPU and disk memory
  - Traversal must stop once these memories are filled

Search Space Compute Schedule



(b) Zig-zag block schedule

Proposed Approach: Zig-zag block schedule:

- A compromise between row-by-row and column-by-column traversals.
- Balances I/O cost and memory usage by alternating traversal directions (zig-zag).
- Block size = (GPU batch size) x (#GPU batches)

Search Space Compute Schedule

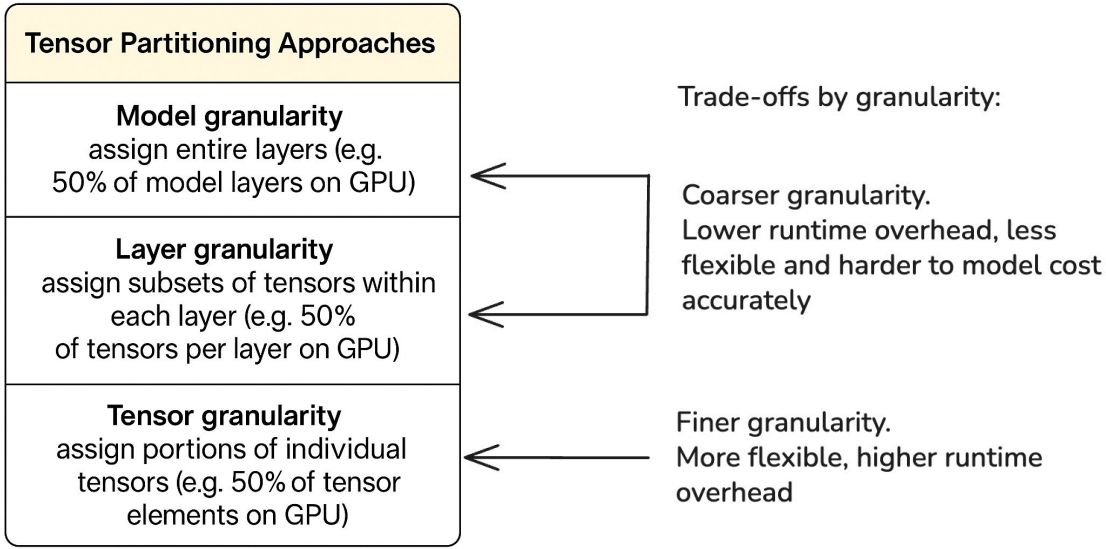
Algorithm 1 Block Schedule with Overlapping

```
for i = 1 to generation_length do
  for j = 1 to num_layers do
    // Compute a block with multiple GPU batches
    for k = 1 to num_GPU_batches do
      // Load the weight of the next layer
      load_weight(i, j + 1, k)
      // Store the cache and activation of the prev batch
      store_activation(i, j, k − 1)
      store_cache(i, j, k − 1)
      // Load the cache and activation of the next batch
      load_cache(i, j, k + 1)
      load_activation(i, j, k + 1)
      // Compute this batch
      compute(i, j, k)
      // Synchronize all devices
      synchronize()
    end for
  end for
end for
```

Goal: Make the high cost of I/O operations (moving data between the disk, CPU, and GPU) amortized by spreading that cost over a large number of computations

- Overlap operations across batches and layers:
  - Loads, stores, and computation
- Implemented via block scheduling (Algorithm 1)
- Rely on operating systems and CUDA drivers to resolve the schedule of the underlying hardware resources

Search Space Tensor Placement

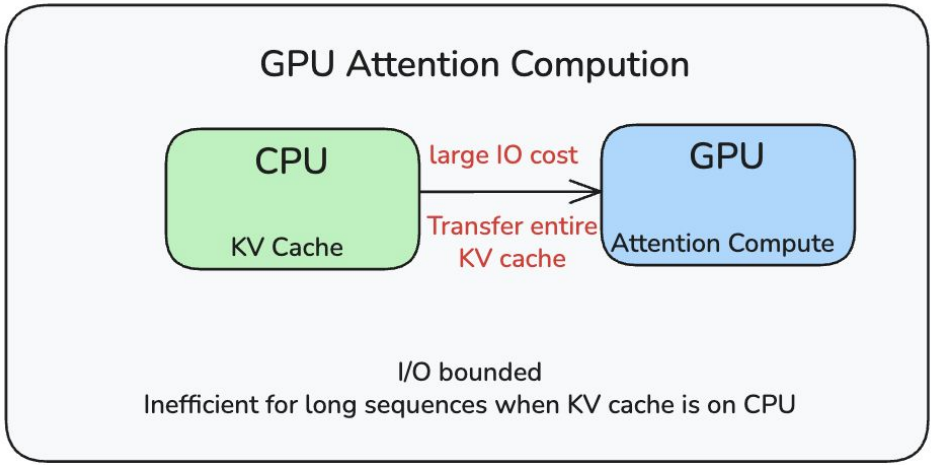


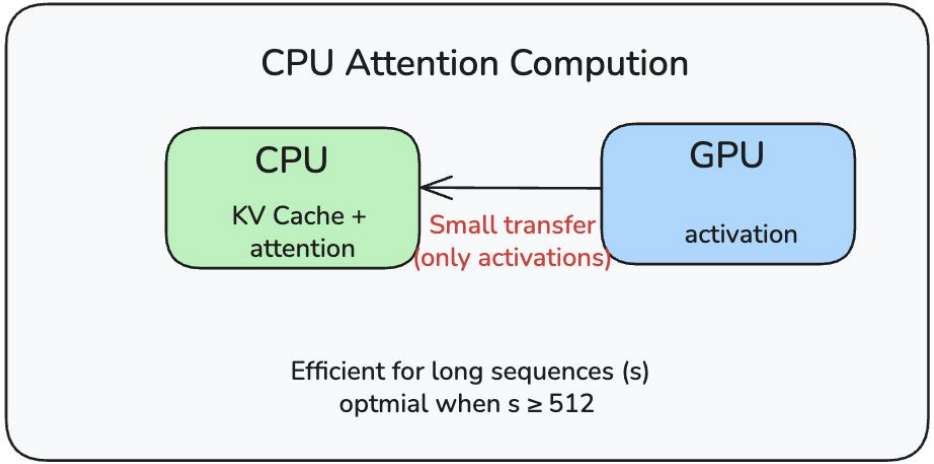
Chosen design in FlexGen:

- Layer granularity is used for weights (balances overhead and flexibility)
- Tensor granularity is used for activations and the KV cache (to maximize placement flexibility)

Search Space Computation Delegation

During decoding, the computation of attention scores is often I/O-bounded rather than compute-bounded





FlexGen Overview

Offloading (moving tensors in and out of memory hierarchy)

- Construct a search space
  - Compute Schedule
  - Tensor Placement
- **Cost model + Policy Search**

Cost Model

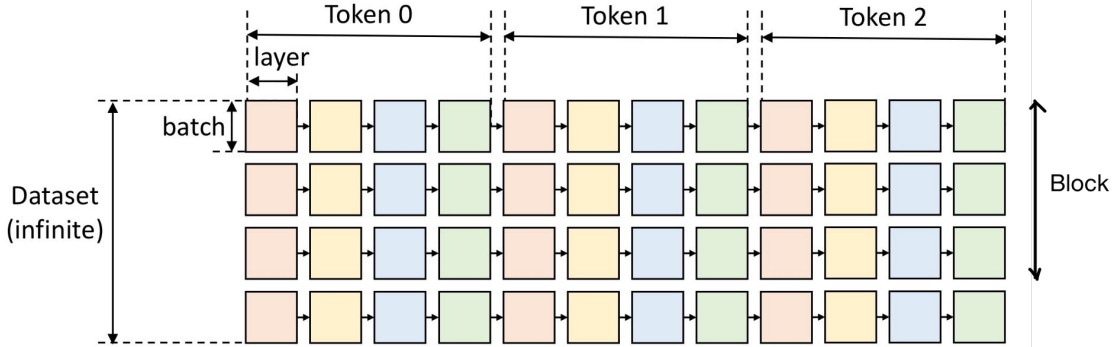


Figure 2. Computational graph of LLM inference.

*legend: h1 = hidden size, h2 = MLP hidden size, bls = block size,
s = prompt sequence length, n = output sequence length

Weights (per layer)
Size = 8h^2 + 4h1 * h2
bytes

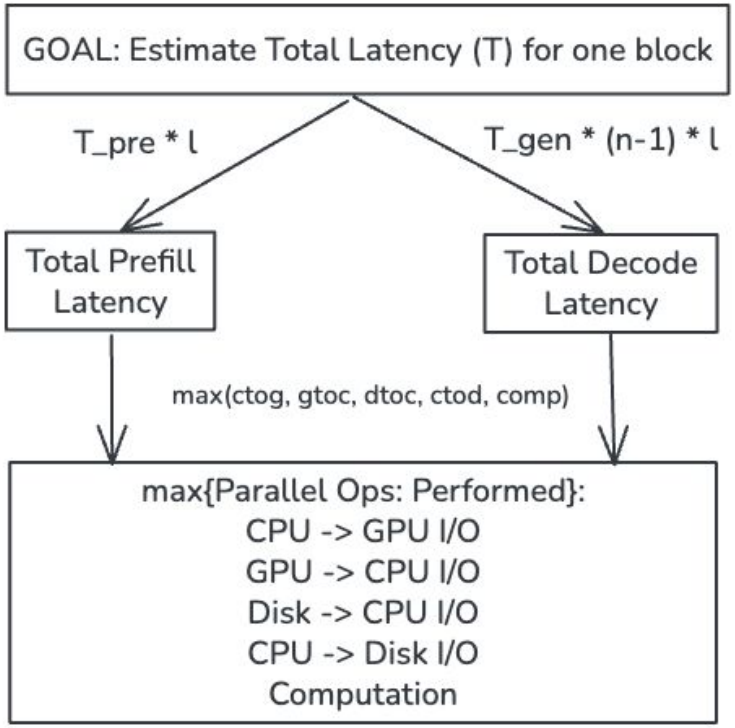
KV Cache (per layer)
Size = 4 * bls * (s + n/2) * h1

Activations (per layer)
Size: 2 * bls * h1

dtoc_g = 1 / (disk_to_cpu_bandwidth) * Total_Data_Size
where Total_Data_Size = Weights + KV Cache + Activations

GPU computation terms are done similarly by summing up all computation events

Also estimates the peak memory usage on CPU, GPU, Disk for that strategy and uses them as memory constraints



Policy Search

Enumerate Batch Size Configurations → Solve Linear Programming → Select Best Policy Combination

Obse
-
-

GPU
4,8

gbs=4, bls

Problem

A computer cannot just split a model's weights "arbitrarily". A model is made of distinct separate layers →can't just tell the computer to "put 75.3% of the layers on the GPU." This becomes a discrete optimization problem and runs much slower.

Solution

1) Brute force those that we know have bounded constraints (eg. gbs, bls)
2) Relax the problem of accurately modeling peak memory usage to reduce it into a linear programming problem (Polynomial time)

Consequences

Sometimes, the "optimal" policy the search finds will still crash with an Out-Of-Memory (OOM) error when you try to run it. In this case, a human just "manually adjusts the policy slightly" to make it fit.

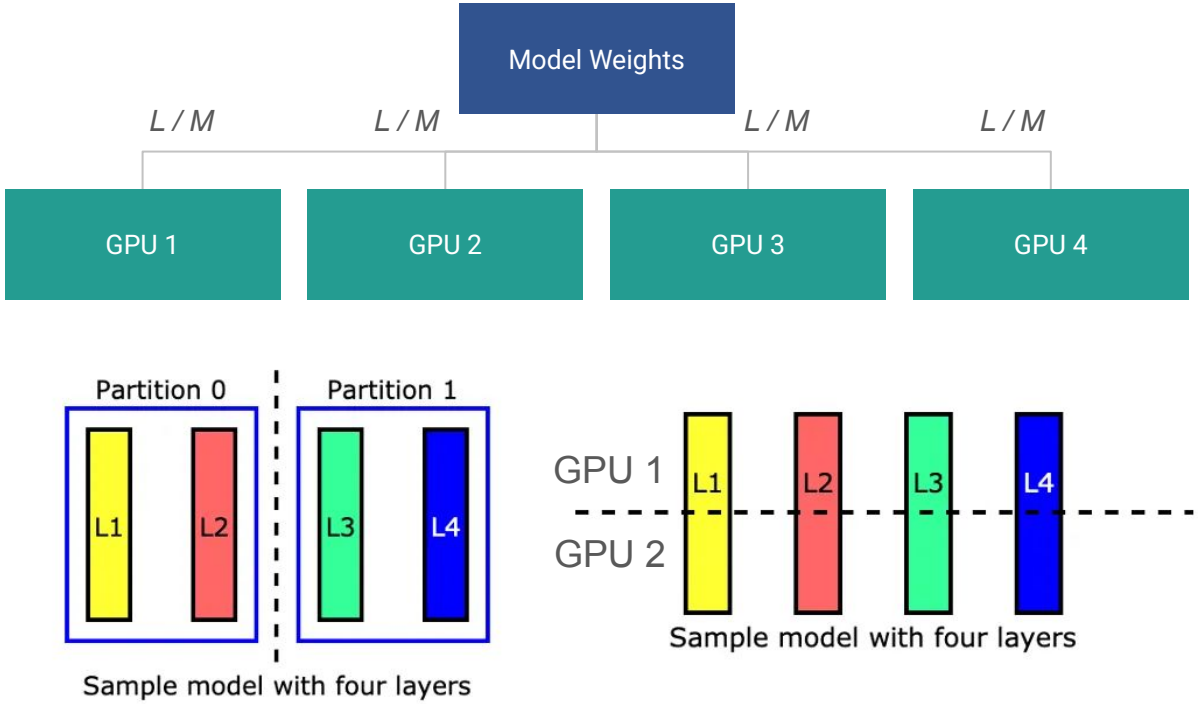
Key Idea: The cost model gets you 99% of the way to the best solution, but a final bit of manual tuning might be needed.

$$
\begin{aligned}
\text{disk peak memory} &< \text{disk mem capacity} \\
wg + wc + wd &= 1 \\
cg + cc + cd &= 1 \\
hg + hc + hd &= 1
\end{aligned}
\tag{1}
$$

cg, cc, cd (% of KV cache placed on GPU, CPU, Disk respectively)

hg, hc, hd (% of activations on GPU, CPU, Disk respectively)

Extension to Multiple GPUs



Pipeline Parallelism vs Tensor Parallelism

The single-GPU strategy is still heavily limited by I/O and has a low GPU utilization. FlexGen uses pipeline parallelism when given multiple GPUs to reduce the memory pressure on each individual GPU.

	Pipeline	Tensor
Better for	Throughput	Latency
Communication Overhead	Low	High

A model with L layers is equally partitioned across M GPUs. Each GPU is now only responsible for a small, L/M-layer chunk, reducing the multi-GPU problem back into a single-GPU problem → can just directly reuse the exact same single-GPU policy search.

Results

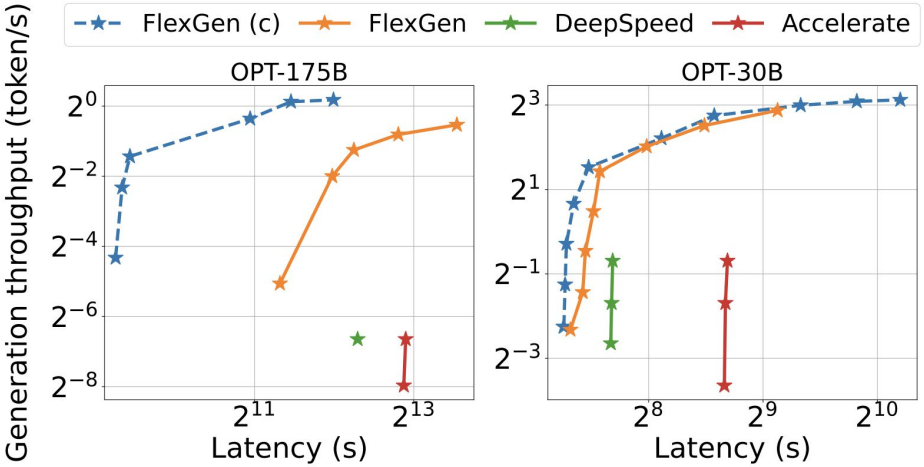


Table 1. Hardware Specs

Device	Model	Memory
GPU	NVIDIA T4	16 GB
CPU	Intel Xeon @ 2.00GHz	208 GB
Disk	Cloud default SSD (NVMe)	1.5 TB

- Compared 3 offloading systems for OPT-175B and OPT-30B
- Ran on synthetic datasets where all prompts are padded to the same length (512 and 1024) with a fixed 32 output tokens for each prompt
- Higher latency requirements enable increased batch sizes which improves throughput (40x higher throughput on 5000s and 69x higher throughput on 12000s)
- Up to 100× improvement by using an effective batch size of 144.
- Others' batch sizes limited by out-of-memory issues

Weaknesses

- FlexGen was evaluated for the latency throughput tradeoff on a single GPU; however, the difference diminishes with multiple GPUs, which the paper did not evaluate
- If most sequences have similar lengths, batching efficiency is high, but if some are very long and others short, FlexGen wastes time on useless computation of padding tokens (common pattern in sparse datasets/workloads)
- It was run on a system with 208 GB of CPU RAM, which is higher than typical consumer grade CPUs

Aspect	FlexGen	PowerInfer
Goal	High-throughput batch processing	Low-latency interactive inference
Hardware	Single GPU (16GB T4) + CPU/Disk	Consumer GPU (RTX 4090) + CPU
Core Innovation	3-tier memory hierarchy optimization	Exploits neuron activation locality
Strategy	Offloading strategy for tensor placement	Hot/cold neuron partitioning via ILP
Execution	Zig-zag schedule for weight reuse	GPU-CPU hybrid with adaptive prediction
Key Optimizations	• 4-bit compression • I/O-compute overlap • CPU offloading	• Adaptive neuron predictors • Sparse operators • Selective activation
Performance	100x throughput vs baselines (1 token/s on OPT-175B)	11.69x speedup vs llama.cpp (82% of A100 performance)